Compiling Against Intent

OR MAGEN, Independent Researcher, USA

Source programs mix invariant requirements with incidental choices, and an optimizing compiler bound by the as-if rule must preserve both. The set of programs that satisfy the requirements is often larger than the set of as-if variants of a correct baseline, so as-if search may omit a cheaper legal program. A queue whose requirements are FIFO order and capacity at least 1000 may be written with 1000 cells and remainder indexing. A 1024-cell ring indexed by a mask still satisfies those requirements, and as-if cannot emit that ring from the baseline. That set cannot be searched as a whole.

IntentCompile is a restricted compiler that starts from an existing baseline and a checkable contract naming those requirements, generates candidates, admits those a sound verifier proves, and ranks them by measurement. IntC is a C instance: a rewrite catalog, CBMC on bounded harnesses, and MSVC `/O2`. On that baseline, `/O2` keeps 1000 cells and remainder indexing, and IntC admits the 1024-cell program and selects it at 2.3× on the named benchmark.

1 Introduction

An optimizing compiler may replace a program with any program that has the same observable behavior, commonly referred to as the as-if rule of ISO C++ [20]. That rule lets compilers reorder, inline, vectorize, and eliminate code, and it makes the source the complete specification. Every detail the source fixes is one the compiler must preserve, whether the developer cared about it or not.

Source code mixes two kinds of decisions. Domain invariants are what the program must actually guarantee, such as packets leaving a queue in the order they arrived and the queue holding at least a thousand packets before it drops. Incidental choices are details the code had to fix in order to run, such as a capacity of exactly 1000 or a remainder instruction that maps a counter onto that capacity. A packet queue written for bursts of 800 packets with 25% headroom needs at least 1000 slots. The developer writes 1000, and each enqueue computes `slot = i % 1000`, which compiles to a chain of multiplications and shifts. With 1024 slots the same index is a bitwise `and`. Both sizes meet the requirement, and the as-if rule allows only the size in the source, because the two queues disagree once they hold 1000 packets.

Developers split these decisions by hand. Folly's single-producer queue rounds its backing array up to a power of two and indexes with a mask, while `isFull` still respects the capacity the caller asked for [35]. DPDK's exact-size rings do the same separation [12]. ClickHouse sizes hash tables in powers of two and places keys with a mask [8]. Those libraries write the split into the source, and a compiler bound by as-if cannot introduce it from a baseline that used 1000 and a remainder. This paper asks for that compiler.

Compiling against a specification instead of the source is an old idea. Leroy defines compilers that preserve a specification rather than exact behavior [28]. Refinement calculi derive implementations from specifications [3, 39]. Fiat, Cozy, and Primrose choose among the implementations a specification admits, Primrose as annotations on Rust [11, 29, 45]. STOKE optimizes floating-point code against a stated error bound rather than equivalence [51]. This paper asks what that idea is, as a compilation problem for ordinary systems code, and what an automated compiler can actually do with it.

The answer has three layers.

The Ideal Problem. The developer states a functional contract Φ_{func} over observable behavior, including named implementation parameters such as a capacity of at least 1000, together with

Author's Contact Information: Or Magen, Independent Researcher, USA.

a resource budget and a cost objective. Programs that satisfy Φ_{func} are legal, and those that also satisfy the budget are feasible. The ideal compiler returns a cheapest feasible legal program. That legal set contains every as-if variant of a correct baseline, and more programs exactly where Φ_{func} leaves a decision open.

A Restricted Compiler. IntentCompile proposes candidates from a generator (rewrites and parameter holes), admits those a sound verifier proves within a time budget, and hands the winner to an unmodified production compiler. What it can use is the verified fragment of the generated set, and it emits P_1 or a program cheaper on the supplied O.

An Instance. IntC is a C instance of IntentCompile. It takes a C baseline, a contract written as parameter constraints plus CBMC harnesses, and an optional benchmark. It rewrites remainder, division, and multiplication by a parameter. It also applies representation rewrites: pad-and-mask, which grows a private store while keeping API capacity, and structure-of-arrays. Arithmetic rewrites drop a sqrt from a disk test or a double accumulator from an inner product. It fills parameter holes by search near the baseline, admits candidates a bounded model checker proves, and selects by measurement. On the FIFO ring, that winner is the 1024-cell mask, 2.3× on the named benchmark. What it claims of an admitted program is $\models_K$: the harness holds on every execution the bound covers. It emits reviewable source so a developer can inspect an admitted change as C, including a change that disagrees with the baseline on an observable result.

Contributions.

- The compilation problem: contracts with named implementation parameters, four kinds of slack, and an ideal objective over the legal set (§3).
- A restricted compiler whose search space is a generator, whose admission is a sound verifier, including the bounded judgment $\models_K$, and whose backend is an unmodified production compiler (§4).
- The IntentCompile pipeline and IntC, a C instance of it (§5, §6).
- Evaluation of IntC (§7): whether it runs on extracts from real-world projects, whether as-if compilation and the internals behave as claimed, and whether ranking under the same `/O2` emits a cheaper program on the named driver or keeps P_1. On the ring that driver is 2.3×.

Section 2 walks through the ring buffer and what existing tools can do with it. Sections 3–4 are the two theory layers. Sections 5–6 are the algorithm and the artifact. Section 7 answers the three questions of IntC. Section 8 places the work. Section 9 names the risks that remain. Section 10 closes.

2 Overview

2.1 Motivating Example

A network ingress path buffers packets in a ring so that bursts do not cause drops. Bursts reach 800 packets, and the developer provisions 25% headroom, so the ring needs at least 1000 slots. The queue keeps two free-running 64-bit counters. `tail` counts enqueued packets and `head` counts dequeued ones. The queue holds `tail - head` packets. Each counter value maps to a storage slot by taking it modulo the capacity.

```
#define CAPACITY 1000
uint64_t ring[CAPACITY];
uint64_t head = 0, tail = 0;

bool enqueue(const uint64_t pkt) {
```

```
    if (tail - head == CAPACITY) return false;
    ring[tail % CAPACITY] = pkt;
    ++tail;
    return true;
}
```

Every enqueue and dequeue computes a remainder by 1000. MSVC 19.29 at `/O2` on x86-64 compiles that remainder to a reciprocal multiply:

```
; i % 1000
mov  rax, 442721857769029239
mul  rcx
mov  rax, rcx
sub  rax, rdx
shr  rax, 1
add  rax, rdx
shr  rax, 9
imul rax, rax, 1000
sub  rcx, rax
mov  rax, rcx
ret
```

A 1024-cell ring uses `i % 1024`, which is the same as `i & 1023`, and the same compiler emits a mask:

```
; i % 1024
and  ecx, 1023
mov  eax, ecx
ret
```

Table 1 splits the requirement, the source, what an as-if compiler may emit, and what a contract admits. As-if will not rewrite 1000 to 1024, because the two queues disagree on the 1001st consecutive enqueue from empty. In isolation, on an Intel Core i7-14700K, a dependent chain of slot computations takes a median of 2.67 ns per step with capacity 1000 and 0.39 ns with capacity 1024, a factor of 6.8. When the computations are independent, the factor is 3.1. These figures measure the slot computation alone. Whole queues, where memory traffic dilutes the difference, are measured in Section 7, and the larger ring uses 24 more slots.

Both capacities meet a requirement of at least 1000 slots. Which is better depends on the deployment. On a server where the path is latency-critical and memory is plentiful, 1024 is the better choice. On a microcontroller whose memory budget is exactly the 1000-slot footprint, the extra 24 slots do not fit. The preference is specific to the target, so neither capacity is right in the source as a universal constant.

That change is not the only way to use a power-of-two backing store. A library can allocate 1024 slots and still report full at 1000, using the extra slots only so that indexing is a mask. Folly's queue after September 2026 does exactly that: it rounds storage up internally and keeps the caller-facing capacity [35]. The two designs use the same hardware trick and are different contracts: one changes when enqueue fails, and the other does not.

2.2 What Existing Tools Can Do

Conventional Compilers and Translation Validation. The two queues that become full at different lengths are not observationally equivalent. Translation validators such as Alive2 [30] check

Table 1. Columns are the requirement, the source P_1, what MSVC `/O2` emits under as-if, and a program the contract admits. As-if must keep the remainder that is in the source. A contract with `CAPACITY` $\geq$ 1000 admits 1024.

Requirement	Source P_1	MSVC `/O2` (as-if)	Contract admits
`CAPACITY` $\geq$ 1000, FIFO, full at `CAPACITY`	`#define CAPACITY 1000; i % CAPACITY`	reciprocal multiply for `% 1000`; 1001st enqueue fails	`CAPACITY 1024; i & 1023`; 1001st enqueue succeeds
full at 1000; backing store not in the API	`CAPACITY` and `STORAGE` both 1000; `i % STORAGE`	keeps remainder-by-1000	`STORAGE 1024, CAPACITY 1000`; mask; enqueue still fails at 1000

refinement against the source, which allows extra freedom only where the source has undefined behavior, and this queue has none on its intended inputs. Alive2 rejects the 1024-cell ring against the 1000-cell source: the two disagree on the 1001st enqueue from empty.

Compiler Flags. Production compilers already relax as-if under flags, and `-ffast-math` is one example: it licenses reassociation and other algebraic rewrites that IEEE-754 would forbid, and the waiver can be scoped with pragmas. Those rewrites change observable numeric behavior, including treating NaN and infinities as unused, sometimes with measurable implications, but they do not let the compiler replace a 1000-cell remainder with a 1024-cell mask.

Compiler Data-Layout Transformations. Production compilers do change data layout. IBM's XL compilers reshape arrays of structures [62], HP's compilers split and reorder structures [18], and automatic pool allocation regroups heap objects [25]. These transformations must remain invisible to the program. They are often abandoned when the analysis cannot prove that [18], and changing a logical capacity is visible by construction, so it is outside their reach. Padding a private array while preserving the API is closer to what they already do, and still requires proving that the extra cells are never observed.

Annotations and Assumptions. C++'s `[[assume]]` states facts the compiler may rely on. A false assumption is undefined behavior, and the C++26 contracts design leaves out an assume semantic for that reason [4]. Recent Clang attributes permit array-of-structures to structure-of-arrays conversions on annotated code [46]. Both are trusted rather than checked, and neither states that a capacity may grow.

Scheduling Languages. Halide and Exo separate an algorithm from its schedule [19, 47]. The algorithm's meaning, including the size of the ring, stays fixed.

Specification-Based Synthesis. Cozy synthesizes collection implementations from specifications, and Primrose selects container implementations that satisfy stated properties [29, 45]. Fiat refines specifications into verified implementations in Coq [11]. These systems accept freedom from a specification, in their own languages or libraries. Primrose comes closest to existing code: it reads annotated Rust and selects a library implementation. This paper starts from an existing C program, a contract over its observable behavior, and that language's production compiler.

2.3 Intent Compilation on the Example

The developer states the requirement instead of a single capacity. In IntC that is a YAML file plus C harnesses. The contract has four pieces: a constraint on named parameters, a functional harness, an optional resource budget, and a measured objective. On this queue:

```
/* CAPACITY >= 1000 */
/* FIFO; enqueue fails iff the ring holds CAPACITY packets */
```

```
/* optional static-memory ceiling */
/* measured ns/op on a given target */
```

IntentCompile (§5) rewrites `e % CAPACITY` to `((e) & (CAPACITY - 1))`, which requires a power-of-two capacity, searches for the admissible power of two closest to 1000, and asks a bounded model checker whether every sequence of at most K operations from an empty queue satisfies the harness (the bound of $\models_K$, §4.2). Under a server profile with no tight memory budget, 1024 is admitted and, if it measures faster, emitted as source. Under an 8016-byte static-memory ceiling (the 1000-slot baseline's footprint), 1024 fails the budget at 8208 bytes and the baseline is kept. It also tries a pad-and-mask form: a `STORAGE` hole, a mask on that store, and fullness still at `CAPACITY`. That candidate is the unexposed-slack case of Section 3.3. When the baseline already names both parameters, the mask rewrite on `STORAGE` is enough.

3 The Ideal Problem

This section defines the compilation problem: a contract over observable behavior, the programs that satisfy it, and an objective that picks a cheapest one. Leroy's preservation of a specification [28, §2.1] and the refinement calculi [3, 39] are the setting. Named implementation parameters, a taxonomy of slack, that objective, and then a split between the functional contract and the optimization contract make the problem specific enough to compile against.

The definitions apply to any language with a deterministic sequential semantics and a notion of observable behavior. The running examples are in C, and the as-if rule and undefined behavior are the same in C++.

3.1 Programs and the As-If Rule

Let Σ be the set of initial states of a program, covering its inputs and initial memory. Running program P on $\sigma \in \Sigma$ has one of three results. It may terminate in a final state σ' after producing a trace $\tau \in \mathcal{T}$ of observable events, written $\langle P, \sigma\rangle \Downarrow (\sigma', \tau)$. It may run forever, written div. Or it may reach an operation whose meaning the language leaves undefined, written ub. We write $[\![P]\!](\sigma)$ for the result.

In C and C++, the observable behavior consists of I/O and accesses to volatile objects [20, §4.1.2]. Local values, the order of intermediate computations, and unescaped layout are not observable, so comparisons use only those observable parts.

Definition 3.1 (Observational Equivalence). $P_1 \approx P_2$ iff $[\![P_1]\!](\sigma)$ and $[\![P_2]\!](\sigma)$ have the same observable part for every $\sigma \in \Sigma$. We write $[P]_\approx$ for the set of programs observationally equivalent to P.

$[P]_\approx$ is the space a conventional optimizing compiler may search. Undefined behavior loosens this slightly: on inputs where the source is undefined, the compiled program may do anything [26, 30].

3.2 Named Parameters and Functional Contracts

Implementation parameters are named constants in the source, written θ_P. The set of names is part of the contract. Extracting θ_P is syntactic: a `#define` capacity, not a property of $[\![P]\!]$. Two observationally equivalent programs may differ in θ. The contract is indexed by θ, so $Post_\theta$ can depend on the capacity the implementation chose.

Definition 3.2 (Functional Intent Contract). A functional intent contract is a triple

$$\Phi_{\text{func}} = \langle Pre, Post, \mathcal{I}\rangle$$

where $Pre \subseteq \Sigma$ is the set of initial states the program must handle, $\mathcal{I}$ is the set of acceptable parameter valuations, and $Post$ gives, for each $\theta \in \mathcal{I}$, a relation $Post_\theta$ between an initial state and

the acceptable observable results. Program P satisfies Φ_{func}, written $P \models \Phi_{\text{func}}$, iff $\theta_P \in \mathcal{I}$ and

$$\forall \sigma \in \mathit{Pre}.\ \exists \sigma', \tau.\ \langle P, \sigma \rangle \Downarrow (\sigma', \tau) \ \wedge\ (\sigma, (\sigma', \tau)) \in \mathit{Post}_{\theta_P}.$$

The triple follows Design by Contract [36]. Satisfaction requires termination without undefined behavior on every input in *Pre*, and the contract mentions only observable results. The set of satisfying programs is $\mathrm{Sat}(\Phi_{\text{func}}) = \{P \mid P \models \Phi_{\text{func}}\}$. We write $P_1 \sim_{\Phi_{\text{func}}} P_2$ when both lie in that set.

Each component states one kind of intent. *Pre* states which inputs matter. $\mathcal{I}$ states acceptable parameter valuations, so a queue that must absorb bursts of 800 packets with 25% headroom has $\mathcal{I} = \{\theta \mid \theta_{\text{cap}} \geq 1000\}$. Post_θ states what the program must do, relative to the parameters it chose: enqueued packets come out in order, and an enqueue fails exactly when θ_{cap} packets are stored.

Definition 3.3 (Realizability). Φ_{func} is vacuous if $\mathit{Pre} = \emptyset$. It is contradictory if no parameter valuation, input, and result jointly satisfy *Post*. It is realizable if there exists $\theta \in \mathcal{I}$ such that every $\sigma \in \mathit{Pre}$ has at least one acceptable result under Post_θ.

Vacuous contracts are rejected before search: they have empty *Pre*. A contract whose *Post* no program can satisfy is discovered later, when no candidate is proved. Realizability is the $\forall\sigma\,\exists o$ condition.

3.3 Slack

$\mathrm{Sat}(\Phi_{\text{func}})$ differs from $[P]_\approx$ where the contract leaves a decision open. There are four ways to leave one open.

- *Input slack.* $\mathit{Pre} \neq \Sigma$. On excluded inputs any behavior is acceptable. This is the kind undefined behavior already provides, and the kind SeGaBench grants by fixing a valid-input domain [21].
- *Output slack.* For some covered input, Post_θ accepts two distinct observable results. A process-ID allocator must return an unused ID, or `NONE` when every ID is in use. Scanning from ID 1, and continuing from a cursor, both satisfy that, and they return different IDs once an ID below the cursor has been freed. No conventional compiler may replace one by the other.
- *Logical-parameter slack.* The contract accepts several parameter valuations, and the choice is visible in the results. Queues of capacity 1000 and 1024 reject a full-queue enqueue at different points. Both are acceptable when $\mathcal{I}$ requires only a capacity of at least 1000.
- *Unexposed slack.* Extra resources or a different representation that the interface does not expose. A queue that allocates 1024 slots and still becomes full at 1000 uses a power-of-two mask while preserving caller-facing capacity. Folly and DPDK's exact-size rings are this case [12, 35]. If the extra cells and the mask are unobservable at the API, the as-if rule may already permit the change, provided the analysis can see it. The contract still has to license the extra memory, which is a budget concern (§3.5).

Input and output slack are don't-cares in Karnaugh's sense: an incompletely specified function can be minimized by assigning combinations the specification leaves open [23]. Logical-parameter slack and unexposed slack concern the implementation rather than unspecified I/O. The first is a named parameter visible in the results, and the second is extra store or a representation the interface does not name.

The four kinds of slack are properties of Φ_{func}. Whether $\mathrm{Sat}(\Phi_{\text{func}})$ is larger than $[P]_\approx$ is settled by local patching.

A language supports *local patching* if for every program P, state σ_0, and producible observable result o, some program behaves like P everywhere except at σ_0, where it produces o. Any language

that can test its input against a constant has this property, including essentially every general-purpose language in use today.

Proposition 3.4 (Legal Set Expands Exactly on Observable Slack). *Assume local patching, and that each $Post_\theta$ with $\theta \in \mathcal{I}$ is realized by some program. Let $P \models \Phi_{func}$. Then $\mathrm{Sat}(\Phi_{func})$ contains a program that is not observationally equivalent to P iff the contract has input, output, or logical-parameter slack, counting only results some program produces. Unexposed slack is not in that list: it need not change those results.*

Proof sketch. If the contract accepts a result on some input that differs from P's, patching produces a legal program outside $[P]_\approx$. Without slack, every covered input has one acceptable result, the same for every allowed parameter, and every input is covered, so every legal program matches P. (Appendix A.)

When the contract has no input, output, or logical-parameter slack, intent compilation is conventional compilation: the approach generalizes the as-if rule rather than replacing it. Unexposed slack can still change cost and memory. The proposition does not say that any extra legal program is faster.

3.4 The Ideal Objective

Let $\mathcal{L}_{\text{Target}}$ be an unmodified production compiler for the target. It implements the as-if rule: idealized, it searches $[P]_\approx$. Let $O(\text{Bin}, \text{Target})$ rank binaries by cost on that target. A conventional compiler, idealized, returns a cheapest binary in the as-if class of P_1:

$$\text{Bin}^*_{\text{trad}} \in \arg \min_{\text{Bin} \in \{\, \mathcal{L}_{\text{Target}}(P') \mid P' \in [P_1]_\approx \,\}} O(\text{Bin}, \text{Target}). \tag{1}$$

An intent compiler wants the same ranking over every binary that satisfies the contract:

$$\text{Bin}^*_{\text{intent}} \in \arg \min_{\text{Bin} \models \Phi_{\text{func}}} O(\text{Bin}, \text{Target}). \tag{2}$$

No compiler searches either set as a whole. Conventional compilation applies heuristic rewrites inside $[P_1]_\approx$, the passes of $\mathcal{L}_{\text{Target}}$, and may never reach $\text{Bin}^*_{\text{trad}}$. Intent compilation has the same gap on a larger set, and splits (2) into a source step and an as-if step:

$$P^\star \in \arg \min_{P' \models \Phi_{\text{func}}} O(\mathcal{L}_{\text{Target}}(P'), \text{Target}), \tag{3}$$

$$\text{Bin}^*_{\text{intent}} \in \arg \min_{\text{Bin} \in \{\, \mathcal{L}_{\text{Target}}(P'') \mid P'' \in [P^\star]_\approx \,\}} O(\text{Bin}, \text{Target}). \tag{4}$$

The first level rewrites the source toward a cheaper program in $\mathrm{Sat}(\Phi_{\text{func}})$, and the second is (1) with $P^\star$ in place of P_1. That composition does not guarantee a minimum of (2), but when P_1 satisfies the contract it is never worse than as-if compilation of P_1, because the first level searches a set that contains every as-if variant of P_1. It separates program-to-program editing under the contract from semantic lowering of a single source, and the second level still yields a contract-satisfying binary from a contract-satisfying source.

Proposition 3.5 (As-If Programs Remain Legal). *If $P \models \Phi_{func}$, $P' \approx P$, and $\theta_{P'} = \theta_P$, then $P' \models \Phi_{func}$.*

Proof sketch. A violation by P' would be a bad parameter or a bad observable result on some admissible input. The parameters match, so the first case is impossible. Observational equivalence transfers the second case to P, contradicting $P \models \Phi_{\text{func}}$. (Appendix A.)

The side condition $\theta_{P'} = \theta_P$ is required because *Post* is indexed by θ. Hence the second level preserves the contract: if $P^\star \models \Phi_{\text{func}}$, every program in $[P^\star]_\approx$ with the same θ does too.

3.5 Optimization Contracts

The source step (3) and the as-if step (4) are not ranked by the same objective, even though both displays used O. The as-if step is $\mathcal{L}_{\text{Target}}$'s search inside $[P^\star]_\approx$. The source step compares programs that need not be observationally equivalent, which that backend does not rank. A ranking for (3) has to be supplied as part of the intent.

That ranking searches a larger set than $[P_1]_\approx$. A rewrite in that set can use more resources than P_1, exceeding what the developer allocated for the program, for instance a larger memory footprint. Those allocations are implicit in P_1, so as-if cannot raise them and $\text{Sat}(\Phi_{\text{func}})$ does not inherit them. Therefore, an operational budget $\mathcal{B}$ is also needed, stating which implementations are valid to search: those whose resource use stays within what the developer is willing to allocate to the program.

Definition 3.6 (Legality and Feasibility). A program P is *legal* iff $P \models \Phi_{\text{func}}$, and *feasible* iff $P \models \mathcal{B}$.

Search is over programs that are legal and feasible. $\mathcal{B}$ and O stay outside Φ_{func} because legality, feasibility, and ranking are different jobs. Φ_{func} states which programs are legal, $\mathcal{B}$ which of those are feasible, and O ranks them. That ranking is supplied as part of the intent. It may be computed from the program text or measured on a profiled run. IntC uses wall-clock time on a named driver. Cyclomatic complexity occupies the same slot (§8). A missed latency target is then suboptimal rather than illegal, and folding the target into the functional contract would reject a legal baseline. Budgets describe the machine and change with the deployment, while Φ_{func} does not. One functional contract can then yield a 1024-slot ring on a server and a 1000-slot ring under a tight memory budget. A check of $\mathcal{B}$ on source approximates a constraint on the compiled binary: padding, alignment, and allocator overhead are decided later.

Adding $\mathcal{B}$ as a constraint on the source step (3) gives

$$P^\star \in \arg\min_{\substack{P' \models \Phi_{\text{func}} \\ P' \models \mathcal{B}}} O(\mathcal{L}_{\text{Target}}(P'), \text{Target}), \qquad \text{Bin}^*_{\text{intent}} = \mathcal{L}_{\text{Target}}(P^\star), \tag{5}$$

where the second equality is (4) once $\mathcal{L}_{\text{Target}}$ has searched $[P^\star]_\approx$, and Section 4 searches a generated subset of that source set.

Definition 3.7 (Intent Specification). An intent specification is a triple

$$\Phi = \langle \Phi_{\text{func}}, \mathcal{B}, O \rangle$$

where Φ_{func} is a functional intent contract (Definition 3.2), $\mathcal{B}$ is a conjunction of operational constraints on a candidate (static memory, a ban on dynamic allocation, alignment), and O is a cost ranking of binaries on a target.

If P_1 already satisfies the contract and the budget, the minimum in (5) is never worse than the one in (1) among programs that keep θ_{P_1}, by Proposition 3.5.

4 A Restricted Intent Compiler

An automated compiler cannot evaluate (5). Membership in $\text{Sat}(\Phi_{\text{func}})$ is undecidable for interesting contracts, by Rice's theorem [48], and even a decidable proof still has a time budget. This section replaces the source step (3) with a generator, a sound verifier V with budget t, and a fallback. The as-if step (4) remains $\mathcal{L}_{\text{Target}}$. The contract bounds what the compiler may do, and what V proves within t bounds what it can use.

4.1 Generator

Let R be a catalog of source-to-source rewrites, each with a syntactic applicability condition and optional side constraints on parameters, for example that capacity is a power of two. Let D be a finite domain for parameter holes, as machine integers are. R and D are inputs of Gen. The *generated set* is

$$\mathrm{Gen}(P_1, R, D) = \{S[h] \mid S \text{ is obtained from } P_1 \text{ by a finite sequence of rewrites from } R,\ h \in D\}.$$

A program is in Gen only if a finite sequence of rewrites from R produces its structure and some $h \in D$ fills its holes, so a cursor-based PID allocator, a linked queue, or a ring that pads storage while keeping caller-facing capacity appears only if R contains that rewrite. Rewrites may change observable behavior, and their outputs are candidates until verified.

4.2 Verifier

A verifier V takes a program, a contract, and a time budget t and answers proved, a counterexample, or unknown. It is *sound* if proved implies $P \models \Phi_{\text{func}}$ under an exact model of machine arithmetic, including overflow.

Many useful verifiers are bounded. A bound K (operations, unwindings) induces a judgment $P \models_K \Phi_{\text{func}}$: the contract holds on every execution the harness covers up to K, and a sound bounded verifier may conclude proved only for that judgment. Timeouts, solver failures, exhausted unwindings, and a property failure on a truncated path are unknown, never proved.

There are two ways to certify a candidate P_2. A relational check proves that P_2 behaves like P_1, taking observational equivalence to P_1 as the bound. The developer writes no Φ_{func} and the query contains both programs. Every candidate outside $[P_1]_\approx$ is then rejected. That query inherits P_1's complexity and may be undecidable, but it is cheap when the programs are structurally close. Those checks remain the right tool for a small patch when no contract is available [30]. A decoupled check proves $P_2 \models \Phi_{\text{func}}$ (or $\models_K$) directly, taking Φ_{func} as the specification. The developer must write that contract, and the query contains only the candidate. This check does not inherit P_1's complexity, but the candidate itself has to lie in a fragment V can decide, or decide up to K. P_1 need not satisfy the contract: it is a starting point for the generator and a fallback. Search stays near that baseline so a candidate is still a program the developer can read against the sources they already have.

This compiler uses the decoupled check, so the *admitted set* at budget t is

$$\mathcal{A}(P_1, \Phi, t) = \{P_2 \in \mathrm{Gen}(P_1, R, D) \mid V(P_2, \Phi_{\text{func}}, t) = \text{proved and } P_2 \text{ passes the check for } \mathcal{B}\}.$$

The two conjuncts are different checks: V admits on Φ_{func}, and a provided checker admits on $\mathcal{B}$, not a second run of V. When V is unbounded and sound, $\mathcal{A}$ is a subset of $\mathrm{Sat}(\Phi_{\text{func}})$. When V is bounded, it is a subset of the K-satisfying programs. Only programs in Gen are submitted to V, and unknown shrinks $\mathcal{A}$ without unsounding an emit. A bounded model checker or another sound oracle can play V, and they differ in which contracts they express and how much of Gen they reach in t. A synthesizer built on V can produce only what V can prove [55].

4.3 Backend

The as-if step (4) is unchanged: the restricted compiler hands a source program to $\mathcal{L}_{\text{Target}}$.

Compiler Correctness Assumption. For every program P and every σ on which P has no undefined behavior, $[\![\mathcal{L}_{\text{Target}}(P)]\!](\sigma)$ and $[\![P]\!](\sigma)$ have the same observable part. That is the as-if rule as compilers implement it, trusted rather than verified.

Proposition 3.5 already says that an as-if program with the same θ remains legal. $\mathcal{L}_{\text{Target}}$ is that as-if on inputs where the source has no undefined behavior, and it keeps θ, so the unbounded case

of the next lemma is that proposition instantiated at $P' = \mathcal{L}_{\text{Target}}(P^\star)$. The bounded judgment is the same fact on the traces $\models_K$ covers.

Lemma 4.1 (Lowering Preserves a Proved Contract). *If $P^* \models \Phi_{func}$ (respectively $\models_K$), then $\mathcal{L}_{Target}(P^*) \models \Phi_{func}$ (respectively $\models_K$).*

Proof sketch. Implementation parameters remain those of the source. P^* has no undefined behavior on inputs in *Pre* (or on the traces $\models_K$ covers), so the compiler correctness assumption applies, and Proposition 3.5 gives the unbounded case. (Appendix A.)

The lemma needs a contract over observable results and a candidate that is UB-free on the covered inputs, so the trusted base is the production compiler, the verifier, and the encoding of the language into the verifier's logic, which is larger than that of a verified compiler or of Fiat's end-to-end proofs [11, 28].

4.4 Selection and Fallback

Winner selection lowers the baseline and every admitted candidate and keeps the binary with the lowest *measured* cost:

$$\text{Bin}^* = \arg \min_{\text{Bin} \in \{\, \mathcal{L}_{\text{Target}}(P) \mid P \in \mathcal{A} \cup \{P_1\} \,\}} O(\text{Bin}, \text{Target}), \tag{6}$$

preferring P_1 on ties. Each admitted source, and P_1, is compiled with $\mathcal{L}_{\text{Target}}$ and ranked by measured cost on the profiled workload and hardware, and the emitted artifact is the winning source, not an object file.

If $\mathcal{A}$ is empty, the compiler emits P_1 unchanged. A change is admitted only after a proof of Φ_{func} (or $\models_K$). Callers whose tests depended on incidental behavior the contract left open can still observe a difference.

4.5 What the Restricted Compiler Guarantees

Theorem 4.2 (Soundness of a Restricted Intent Compiler). *Let V be sound for $\models$ or for $\models_K$, and let the compiler emit source P^* and binary $\text{Bin}^* = \mathcal{L}_{Target}(P^*)$.*

1. *If $P^* \neq P_1$, then P^* is in $\text{Gen}(P_1, R, D)$, V proved it, and it passed $\mathcal{B}$. Hence $P^* \models \Phi_{func}$ or $P^* \models_K \Phi_{func}$, matching V, and the same judgment holds of Bin^* by Lemma 4.1.*
2. *The measured cost of Bin^* is at most that of $\mathcal{L}_{Target}(P_1)$, when measurement runs.*
3. *If $P_1 \models \Phi_{func}$ (or $\models_K$), then Bin^* satisfies the same judgment in every case.*

Proof sketch. Winner selection returns P_1 or a member of $\mathcal{A}$. Membership in $\mathcal{A}$ is part 1. Part 2 is minimization over a set that contains the compiled baseline. Part 3 uses part 1 or Lemma 4.1 on P_1. (Appendix A.)

If P_1 already satisfies the contract, every emit does. If it does not, the compiler replaces it only with a proved program or leaves it as written, so the emit is never less safe than compiling P_1 with $\mathcal{L}_{\text{Target}}$ alone. If some generated hole assignment of a given sketch satisfies the contract and V decides every query, a baseline-directed search that enumerates D in order of distance to θ_{P_1} returns a closest satisfying assignment (Theorem 5.1). If no generated program satisfies the contract, the compiler keeps P_1, and a different structure requires a rewrite in R.

The compiler ranks by O among programs V has proved. How Gen is explored, including any score that orders rewrites, does not change that ranking. Verification cost and execution cost need not agree [49, 60], so the fastest program may lie outside $\mathcal{A}$. They agree on the ring: integer division is expensive on the processor and expensive to bit-blast [41], and the rewrite that removes the remainder also simplifies the proof.

5 The IntentCompile Pipeline

IntentCompile takes a baseline P_1, a specification $\Phi = \langle \Phi_{\text{func}}, \mathcal{B}, O \rangle$, and a target. It builds $\mathcal{A}$ and then applies (6). Appendix B gives each stage as an algorithm.

(1) **CheckRealiz** (Algorithm 1) checks that $\mathcal{I}$ is satisfiable, and reports an unsatisfiable core on conflict. The stage is one solver call and returns when that call does. A timeout is unknown and stops the pipeline. A harness that no program can satisfy is discovered later, when no candidate is proved.

(2) **TheoryLower** (Algorithm 2) applies rewrites from R that strictly decrease a nonnegative integer score on program structure. The score is a weighted count of constructs that make the query expensive for V, so descent aims at a sketch the checker can decide, and the loop terminates because each rewrite drops the score. Crossing the threshold stops that descent. A sketch is still handed to CEGIS when a rewrite applied, even if the score remains above the threshold. If no rewrite applied, there is no TheoryLower seed and GeneratePool still starts from P_1. Outputs are sketches with holes and side constraints. IntC's weights are in Section 6.

(3) **Baseline-Directed CEGIS** (Algorithm 3) fills holes. An optimizer proposes valuations in $\mathcal{I}$ that meet the side constraints and are as close as possible to θ_{P_1}. Holes not in the baseline are minimized subject to those constraints. V checks each candidate. A counterexample refutes that valuation, failing inputs are not constraints on later proposals, and an unknown result stops the search. A finite attempt cap preserves soundness and may miss a farther satisfying assignment. The loop is enumerative CEGIS on a finite hole domain. The same loop fills TheoryLower sketches and GeneratePool sketches. The sketch does not come from the developer, and hole-filling distance is to the baseline so filled holes stay near the program the developer already wrote [9, 54].

(4) **BudgetNormalize** (Algorithm 4) does not generate programs. It applies the provided checker for $\mathcal{B}$ to the CEGIS seeds and drops those that miss $\mathcal{B}$. If none remain, GeneratePool still runs from P_1. P_1 is the fallback of winner selection and is not re-verified.

(5) **GeneratePool** (Algorithm 5) adds further proved programs. It starts from P_1, not from TheoryLower's seeds, so it still runs when TheoryLower is stuck or CEGIS returns nothing. It applies a second catalog of representation rewrites, fills them by CEGIS, and keeps those that V proves and that fit $\mathcal{B}$.

(6) **Winner selection** (Algorithm 6) is the ranking by (6) when the contract names a benchmark. Ranking runs only on P_1 and on programs V proved. When there is no benchmark, the first admitted candidate is emitted (Section 6). The stage emits the chosen program as source, with the proof bound and the measurements, because an admitted change can disagree with P_1 on an observable result and that disagreement has to be readable as source.

Theorem 5.1 (Properties of Baseline-Directed CEGIS). *Let $S[h]$ range over a finite hole domain D, and let N be the attempt cap ($N = |D|$ if there is no cap).*

(1) *If CEGIS returns $S[h]$, then V proved $S[h]$.*
(2) *If V returns on each query, CEGIS returns after at most $\min(N, |D|)$ iterations.*
(3) *If some $h^\circ \in D$ makes $S[h^\circ]$ satisfy the contract (or $\models_K$), V never answers* unknown*, and there is no attempt cap, then CEGIS returns a satisfying assignment closest to θ_{P_1}.*

Proof sketch. Return is only on proved. V respects t, each miss removes one element of D, and the cap is finite, so the loop returns. A satisfying assignment is never excluded, so the first proved valuation is a closest one, unless an attempt cap stops first. (Appendix A.)

The bound $|D|$ is a worst case. Side constraints such as "is a power of two" shrink it. An attempt cap preserves soundness and may drop part 3.

On the ring buffer, $\mathcal{I}$ requires capacity at least 1000. TheoryLower produces a mask sketch with a power-of-two side constraint. CEGIS's closest value is 1024. The verifier sees the masked program and the harness. The remainder in P_1 does not appear in the query.

6 Implementation

IntC is a Python source-to-source compiler for C. Tree-sitter locates spans in the C text, and there is no intermediate representation. Z3 solves parameter constraints, CBMC 6.11.0 is V, and MSVC 19.29 compiles and measures. The cases and the scripts that produce the numbers in Section 7 are available with the paper.

Contracts. Implementation parameters are `#define` constants. $\mathcal{I}$ lists constraints over those names, written `CAPACITY >= 1000`. *Pre* and *Post* are harnesses in C. A harness calls the program with arbitrary inputs, restricts them with assumptions, and asserts observable results [7]. A YAML file records $\mathcal{I}$, $\mathcal{B}$, $\mathcal{O}$, and the paths of those harnesses, and does not generate the asserts. Unwind bounds and timeouts configure V and are not *Post*. The harness sees the candidate's θ through a generated header, which is how $Post_\theta$ depends on θ. An optional compiled-`sizeof` ceiling and a named benchmark complete $\mathcal{B}$ and $\mathcal{O}$.

Verifier. CBMC checks each harness together with out-of-bounds accesses, invalid pointers, signed overflow, division by zero, and invalid shifts. Loops are unrolled with unwinding assertions. A bound that is too small yields `unknown`, not a proof, including when a later property also fails on that truncated path. Arrays use the theory of arrays (`-arrays-uf-always`). Without that encoding, CBMC expands a 1024-element ring element by element and does not finish within 120 seconds even for three operations. Admission is $\models_K$ for the harness bound K.

Stages. The stages are those of Section 5. Only CheckRealiz stops the pipeline: a failed TheoryLower, an empty CEGIS, or an empty budget filter still runs GeneratePool from P_1 and then winner selection.

(1) **CheckRealiz** checks $\mathcal{I}$ with Z3. Unknown on $\mathcal{I}$ fails the stage, as does a name in $\mathcal{I}$ that is not a `#define`.

(2) **TheoryLower**'s score is $10 \cdot n_{\text{rem/div}} + 3 \cdot n_{\text{mul}} + 2 \cdot n_{\text{loop}} + n_{\text{deref}}$, where the four counts are remainders and divisions that are not powers of two, variable–variable multiplies, loops, and pointer dereferences. Power-of-two remainders and divisions, and multiplies by literals, are omitted. The sum is a nonnegative integer, so a rewrite that drops at least one counted construct terminates. The contract supplies the stop threshold, default 2. Crossing it stops that descent. The catalog is three rewrites, and those rewrites introduce the holes.
   - `e % N` becomes `((e) & (N - 1))` when N is a parameter, with side constraint is_pow2(N).
   - `e / D` becomes `((e) » S)` when D and S are parameters, with pow2_pair(D, S).
   - `e * M` becomes a parenthesized left shift of e by a fresh hole S_M with pow2_pair(M, S_M).

   CEGIS sees a TheoryLower sketch only when one of those applied.

(3) **Baseline-directed CEGIS** asks Z3's optimizer for the closest unrefuted valuation of baseline parameters, minimizes extra holes subject to the side constraints, then asks CBMC. A counterexample excludes that valuation and is not a constraint on later proposals. A timeout or an undecided harness is unknown: CEGIS stops and does not add that valuation to the excluded set. The attempt cap is a contract bound whose default is 16.

(4) **BudgetNormalize** sums compiled `sizeof` of file-scope objects, drops a candidate that exceeds the ceiling, and treats a missing ceiling as no check.
(5) **GeneratePool** starts from P_1. Each matching rewrite becomes one sketch $r(P_1)$, filled by the same CEGIS loop. The catalog is general representation rewrites. It is extended with one domain formula, registered the same way a developer would add a formula from their domain.
   - Representation:
     - pad-and-mask (a larger backing array so remainder indexing becomes a mask)
     - narrowing a 32-bit space tag to `uint8_t`
     - array-of-structs to structure-of-arrays
     - aligning a loop bound to a multiple of 4
     - predicating a sparse `if (flag[i])` update
     - 1D halo pad (ghost cells so a 3-point stencil loop does not branch at the ends)
     - replacing `sqrtf(E) < R` by a squared compare
     - narrowing a `double` accumulator to `float`
   - Domain formula: Rec. 601 float MAC to Q15.
(6) **Winner selection** instantiates (6) under `/O2`. It keeps the lowest median of the contract's benchmark, and replaces P_1 only when a candidate is faster by more than that candidate's sample standard deviation. When the contract has no benchmark, the first admitted candidate is emitted without a timing comparison.

Scope. IntC is a C instance of IntentCompile on extracts.

- The contract is `#define` parameters, a YAML record of $\mathcal{I}$, $\mathcal{B}$, and O, and CBMC harnesses, with V bounded CBMC. CheckRealiz decides satisfiability of $\mathcal{I}$, and the implemented conjunct of $\mathcal{B}$ is an optional compiled-`sizeof` ceiling on file-scope objects. Unbounded operation sequences would need an inductive argument.
- The rewrite catalogs are small and can be extended the same way the domain formula is registered. For example, a linked queue or reordering struct fields to cut padding can be added that way. This IntC does not compile general C or C++ programs. For example, C++ atomics, application-scale floats, and storage-format rewrites like CRS-to-ELLPACK are outside it.
- Lowering is MSVC `/O2` on an Intel Core i7-14700K under Windows, and ranking compares P_1 and the admitted programs under that same pair. That pair is O for IntC, and another compiler or machine is another O.

7 Evaluation

This section asks three questions of IntC, in the order of the subsections that answer them.

(1) Does IntC work on code extracted from real-world projects?
(2) Do as-if compilation of P_1, and the oracle, budget filter, and TheoryLower score inside IntentCompile, behave as claimed?
(3) Under the same lowering backend that compiled P_1, does IntC emit a candidate that is cheaper on O than P_1, or the same program?

A last subsection names misses of IntC. Runs use the machine and compilers of Section 6. Tables are emitted by `python reproduce.py` in the artifact (§10). Solver times vary across runs. The admit, timeout, and counterexample outcomes, and the rewrite names, are the claims.

Table 2. One IntC run per row. Columns are the extract, the catalog rule that fired, the harness bound K, CBMC on P_1 and on the admitted program, and how that program compares to P_1.

Case	Rewrite	K	V on P_1	V on winner	As-if
DPDK ring	pow2-mask	4	unknown/120.0s	proved/2.0s	not as-if
DPDK pad	pow2-mask	4	unknown/120.0s	proved/0.6s	unexposed
ClickHouse	pow2-mask	4	unknown/180.2s	proved/63.7s	not as-if
Zephyr	pow2-shift	1	proved/0.5s	proved/0.7s	not as-if
gain	pow2-mul	1	proved/0.1s	proved/0.1s	not as-if
PARSEC	aos-to-soa	6	proved/0.6s	proved/0.6s	layout
Flecs	predicated-update	10	proved/0.1s	proved/0.1s	layout
MachSuite	halo-1d	8	proved/0.1s	proved/0.1s	layout
CMSIS-DSP	align-loop-bound:BLOCK	12	proved/3.3s	proved/7.0s	not as-if
OGRE	narrow-int-field	6	proved/0.1s	proved/0.1s	layout
Box2D	squared-compare	1	proved/13.7s	proved/0.2s	not as-if
Opus	narrow-accum	6	proved/0.7s	proved/12.0s	not as-if
OpenCV	luma-q15	6	unknown/60.0s	proved/0.1s	not as-if
PARSEC fat	aos-to-soa	6	proved/0.6s	proved/0.7s	layout

7.1 Extracts from Real-World Projects

Each row of Table 2 is one run of IntC on a contract that states slack a developer could grant. How each extract was cut from the upstream tree is in the artifact.

Case. The cell names a tagged upstream situation whose hot loop we extracted, except gain, which is a multiply-to-shift written for this catalog. The DPDK ring and pad baselines are the 1000-slot sketches of Section 2.3, not the library sources. ClickHouse's baseline uses 10007 buckets, a prime, not the upstream power of two. PARSEC and PARSEC fat are the same Euler step on four particles, the latter on a 64-byte record that includes four explicit pad fields. CMSIS-DSP is a two-tap FIR.

Rewrite. The cell is the catalog rule that produced the admitted program. TheoryLower contributes remainder-to-mask, division-to-shift, and multiply-to-shift. GeneratePool contributes the representation rewrites and Rec. 601. In table order:

- DPDK ring: remainder-to-mask. CEGIS selects 1024. GeneratePool also admits pad-and-mask into that pool at $K = 4$.
- DPDK pad: remainder-to-mask on `STORAGE`, with `CAPACITY` still 1000. Fullness uses `tail − head` against `CAPACITY`, so enqueue fails at 1000 live packets. The extra 24 cells are a budget cost. TheoryLower and CEGIS produce that mask on the already-named `STORAGE`.
- ClickHouse: remainder-to-mask. $\mathcal{I}$ requires at least 10007 buckets. CEGIS selects 16384, the admissible power of two nearest that floor.
- Zephyr: `tick_us / DIVISOR` to a shift. Relative error at most 2.5% versus `tick/1000` on 32-bit ticks at least 640000, because integer division by 1024 overshoots that bound near 600000. That floor is part of *Pre*. CEGIS selects `DIVISOR` 1024 and `SHIFT` 10.
- Gain: `sample * GAIN` with `GAIN` $\in [8, 16]$ to a left shift by 3. The harness asserts equality with `sample * GAIN` on 16-bit samples, so the admit is not a multiply by the baseline's 10.
- PARSEC: array-of-structs to structure-of-arrays.
- Flecs: predicated update on a 0/1 live flag.
- MachSuite: 1-D halo pad.
- CMSIS-DSP: loop bound aligned to a multiple of 4. $\mathcal{I}$ is `BLOCK` ≥ 5 with search box `BLOCK` ≤ 64; CEGIS selects 8.
- OGRE: 32-bit space tag narrowed to `uint8_t`.
- Box2D: `sqrtf(d2) < R` to `d2 < R*R`. *Post* is the integer disk test $dx^2 + dy^2 < R^2$; the `sqrtf` in P_1 is not that test. The timed bench is 10^6 float pairs, outside $\models_K$.

Table 3. Four switches on the DPDK ring: FIFO oracle, $\models_K$, occupancy-only *Post*, and an 8016-byte ceiling.

Switch	Outcome
Contract oracle (FIFO harness)	proved/0.6s
Admission is $\models_K$	K=4 proved/0.7s; K=10 counterexample/17.3s
Occupancy-only vs FIFO *Post*	capacity-only proved/0.5s; FIFO counterexample/0.4s
BudgetNormalize at 8016 B	emit P1

- Opus: `double` accumulator narrowed to `float`. The harness uses $LEN = 4$ operands in $\{0, 1, 2, 3\}$. The timed bench is $L = 256$ fractional floats, outside $\models_K$.
- OpenCV: Rec. 601 float coefficients to a Q15 shift. The harness is that Q15 formula within one gray level, not the float Rec. 601 sum. $NPX = 4$ in the harness; the timed bench is 1920×1080, outside $\models_K$.
- PARSEC fat: array-of-structs to structure-of-arrays.

***K*.** The cell is the harness bound for $\models_K$: how many operations or loop unwindings CBMC covers on that row (§4.2). Ring, pad, and ClickHouse use four operations from an empty container. Zephyr, gain, and Box2D use one conversion, sample, or compare. The rest are loop unwindings of the extract: 6 on PARSEC's four particles and on OGRE, Opus, OpenCV, and PARSEC fat; 8 on MachSuite; 10 on Flecs; 12 on the two-tap FIR.

***V* on P_1 and on the winner.** Those cells are that CBMC query. `unknown` is a timeout. PARSEC and PARSEC fat prove the Euler step at $n = 4$. CMSIS-DSP proves the two-tap filter. When both programs prove, a longer time on the rewrite is the cost of that query. Table 2's Opus and CMSIS-DSP rows are the two where both prove and the rewrite is the slower query.

As-if. The cell classifies the admitted program against P_1. *not as-if* means a named parameter or a precision choice moved. *unexposed* means the backing store grew and caller-facing capacity did not. *layout* means a representation rewrite the harness treats as unobservable.

7.2 Behavior as Claimed

This subsection answers as-if compilation of P_1, then the oracle, the budget filter, and the TheoryLower score, in that order.

As-if compilation of P_1. The DPDK ring maps a sequence number i to a slot by remainder, and in P_1 that remainder is i mod 1000. MSVC `/O2` still emits a reciprocal multiply for `i % 1000` and `and ecx, 1023` for `i % 1024`, and it does not change 1000 to 1024. A microbenchmark of that slot expression alone uses 15 interleaved trials of $2 \cdot 10^8$ iterations and two seeds. A latency-bound chain measures 2.67 ns versus 0.39 ns, a factor of 6.8, and independent operations measure 0.52 ns versus 0.17 ns, a factor of 3.1. That upper bound has no queue memory traffic.

Table 3 is one switch at a time on that ring's contract for the oracle and the budget.

Oracle. The FIFO harness proves the admitted mask in 0.6 s. Admission is $\models_K$: the colliding mask `& 999` is proved at $K = 4$ and refuted at $K = 10$, because slots collide once nine or more elements are live. An occupancy-only *Post* proves a LIFO stack that the FIFO harness refutes.

Budget filter. BudgetNormalize at 8016 bytes, the 1000-slot baseline: CEGIS still proves the 1024-slot mask at 8208 bytes. Algorithm 4 drops it and the emit is P_1. The 1000-slot remainder still times out at that K.

TheoryLower score. Forty remainder-site variants, on a ring at $K = 4$ and a hash set at $K = 3$, have Spearman's $\rho = 0.11$ between that score (§6) and CBMC time at a 120 s timeout. Every timeout still has a non-power-of-two remainder in the program, and proved times are flat, so a score at or below the threshold is not a decision that V will finish. A score that predicted CBMC time is

Table 4. Contracts that name a `bench.c`, ordered by emit ratio, P_1 last. Columns are the extract, K on the timed harness, CBMC on the admitted program, median wall-clock ns per unit of work of P_1 and of the admit under MSVC `/O2`, and emit.

Case	K	V on admitted	P_1 ns	admitted ns	emit
Box2D	1	proved/0.3s	0.702 ± 0.024	0.108 ± 0.015	6.5×
PARSEC fat	6	proved/0.6s	1.203 ± 0.039	0.346 ± 0.012	3.5×
ClickHouse	4	proved/62.4s	9.455 ± 0.097	4.000 ± 0.043	2.4×
DPDK ring	10	proved/47.8s	1.128 ± 0.017	0.487 ± 0.008	2.3×
Opus	6	proved/11.9s	0.440 ± 0.005	0.273 ± 0.003	1.6×
OpenCV	6	proved/0.1s	0.840 ± 0.011	0.560 ± 0.014	1.5×
CMSIS-DSP	12	proved/6.7s	0.460 ± 0.016	0.927 ± 0.062	P_1

a different score (§9). When TheoryLower finds no remainder or division hole, it plants no seed: PARSEC's Euler step is that case, and GeneratePool still admits structure-of-arrays from P_1.

7.3 Cheaper on O, or P_1

This subsection answers whether ranking under the same lowering backend emits a candidate cheaper on O than P_1, or P_1. Table 4 is the seven contracts that name a `bench.c`. Min, max, and quartiles are in the artifact JSON.

Case. The cell is the Table 2 extract whose timed contract names that driver.

***K*.** The cell is the timed harness bound. Six of the seven match Table 2. The DPDK ring is $K = 10$ here and $K = 4$ in that table. Table 2 used four operations from an empty queue. This row uses ten, the bound that refutes `& 999` in Section 7.2.

***V* on admitted.** The cell is CBMC of the admitted rewrite on that timed harness, including CMSIS-DSP where emit is P_1 and the cell is still the slower `BLOCK = 8` admit. The ring row is that query at $K = 10$. Table 2's ring cell is the same rewrite at $K = 4$. Solver seconds move across runs; the proved/unknown outcome is the claim.

***P*₁ ns and admitted ns.** Those two cells are O: wall-clock nanoseconds per unit of work on that `bench.c` (pair, update, lookup, enqueue, mac, pixel, or FIR block), lower is better. The stage compiles both programs with MSVC `/O2` and takes the median of 31 trials after warmup. A later candidate replaces the current best only if it is faster by more than its sample standard deviation.

Emit. The cell is the ratio of those medians, or P_1 when the admit is not faster by more than its sample standard deviation. In table order:

- Box2D: in-range pairs. MSVC `/O2` still emits `sqrtss` from the baseline, so the squared compare is the ranking win. The listing is in the artifact.
- PARSEC fat: the Euler step at $N = 100000$. The catalog query still proves $n = 4$.
- ClickHouse: fill 4000 keys, then `contains`. The ratio mixes a larger table and a lower load factor with the mask.
- DPDK ring: a dependent enqueue/dequeue chain through a partly full queue. The two programs disagree on the 1001st consecutive enqueue. The isolated slot chain of Section 7.2 overstates that whole-queue gain. The baseline remainder times out at 300 s for $K = 4, 6, 8$ in the recorded sweep, and at 120 s in Table 2.
- Opus: the inner-product kernel.
- OpenCV: luma pixels.
- CMSIS-DSP: one `filter()` call. Extra trips from rounding 5 to 8 are slower, so this row emits P_1. Table 2 used a contract with no `bench.c`, so that row emitted the admitted `BLOCK = 8` rewrite without a timing comparison (§6).

Table 5. Misses of IntC: never generated, budget, slower admit, weak harness, bound too small, driver, and unnamed cost.

Kind	Instance	What happens
Never generated	linked queue, CRS-to-ELLPACK, swap-and-pop	not in the catalog
Budget	1024-slot mask over 8016 B	dropped, emit P_1
Slower admit	FIR `BLOCK` 5⟶8	extra trips, emit P_1
Weak harness	occupancy-only LIFO	FIFO harness refutes it
Bound too small	`& 999`	proved at $K = 4$, refuted at $K = 10$
Driver	slot chain vs ring `bench.c`	6.8× vs 2.3×, same rewrite
Unnamed cost	false sharing	not measured

7.4 Misses of This Instance

This subsection names where the three answers fail on IntC. Table 5 is those misses.

Kind. The cell is how the miss happens.

Instance. The cell is the extract or rewrite.

What happens. The cell is the outcome. In table order:

- Never generated: a linked queue, a CRS-to-ELLPACK layout, and a swap-and-pop compaction are not catalog sites, so they never become candidates. A CRS-to-ELLPACK rewrite that kept the stored values would still stream the extra zeros ELLPACK pads.
- Budget: the 1024-slot mask proves and is 8208 bytes. The 8016-byte ceiling drops it and the emit is P_1 (Section 7.2).
- Slower admit: FIR `BLOCK 8` does extra trips on one `filter()` call, so ranking emits P_1 (Section 7.3).
- Weak harness: an occupancy-only *Post* proves a LIFO stack that the FIFO harness refutes (Section 7.2).
- Bound too small: `& 999` proves at $K = 4$ and is refuted at $K = 10$ (Section 7.2).
- Driver: the isolated slot chain is 6.8×. The ring `bench.c` is 2.3× on the same rewrite (Section 7.2).
- Unnamed cost: concurrent false sharing is not in any named driver, so it cannot reject a candidate.

8 Related Work

Compiling against a Specification. Leroy distinguishes compilers that preserve a program's behavior from compilers that preserve only a specification, and notes that the latter leave more room for optimization [28, §2.1]. Refinement calculi make any program that refines a specification a correct implementation [3, 39]. Fiat refines specifications with deliberate freedom into proved implementations in Coq [11]. Data representation synthesis, Cozy, and Primrose choose among the implementations a specification admits [15, 29, 45].

Those systems compile a specification: Fiat and Cozy in a language built for that compilation, Primrose from annotations on Rust into a library container. IntentCompile uses a general contract with a general-purpose program the developer wrote as the seed of the search. The compiler treats that seed as a proxy of the developer's mental model of the problem and searches from it for a similar but cheaper solution. Section 3 states the problem for a general-purpose language of the kind developers already run in production. IntC is the C instance this paper evaluates. Table 6 places that instance against specification compilers, schedulers, and as-if-relaxing tools.

Table 6. Columns are the host language, whether θ is a named parameter the developer writes, whether the search includes a rewrite as-if would reject, the admission oracle, and whether the compiler can emit P_1 when the search fails. IntentCompile is the algorithm. IntC is the C instance.

System	Language	Named θ	Non-as-if slack	Sound V	Fallback P_1
Fiat [11]	Coq	in the spec	yes	Coq	no
Cozy [29]	spec to Java	in the spec	yes	synthesis checker	no
Primrose [45]	annotated Rust	container spec	yes	SMT	library pick
STOKE [51]	x86 asm	error bound	numeric	validator	best-so-far
Alive2 [30]	LLVM IR	no	no	yes	n/a
Halide/Exo [19, 47]	own language	schedule	no	lowering	n/a
PetaBricks [2]	own language	autotune knobs	authored menu	no	no
IntC	C	$\mathcal{I}$	yes	$\models_K$	yes

Scheduling Languages. Halide and Exo keep a DSL algorithm, including sizes, fixed and search its schedule [19, 47]. IntentCompile searches structured refactors of a general-purpose program, starting from the current implementation and a contract that names which of its choices are slack.

Optimizing under Relaxed Correctness. Production compilers already ship named, opt-in relaxations of as-if. `-ffast-math` is one example, a fixed license to reassociate and otherwise rewrite IEEE-754 arithmetic, and scoped floating-point pragmas are another. STOKE optimizes floating-point code against a user-chosen error bound [51]. Sound compilation of reals and Chisel choose any implementation within a stated tolerance [10, 37]. Conditionally correct superoptimization exploits facts about inputs beyond undefined behavior [52]. Carbin et al. verify acceptability properties of relaxed programs [6]. Loop perforation trades accuracy for speed under empirical checks [53]. PetaBricks lets the developer write several algorithms for one computation and autotunes among them by measurement [2].

They each implement a particular relaxation, an opt-in compiler flag, an error bound, extra input facts, perforation, or an authored menu. Because IntentCompile does not take as-if as the bound, a contract can name several of those freedoms together, and the search is a structured refactor of the current implementation.

Translation Validation and Superoptimization. Translation validation [40, 44] and CompCert [28] certify as-if: CompCert proves that the backend preserves the source's observable behavior. Alive2 checks that a transformation refines its LLVM source [26, 30]. Souper synthesizes peephole optimizations that the same check admits [50]. Recent systems pair language models with these checkers [13, 57]. Rinard finds that agents under relational equivalence can pick slower algorithms because they are easier to prove equivalent [49].

Those checkers ask whether a candidate is equivalent to the source or refines it. IntentCompile keeps the same split of untrusted proposer and sound checker, and the checker asks whether the candidate satisfies the contract, which is what admits a 1024-cell ring from a 1000-cell baseline.

Lifting and Verification-Friendly Programs. Verified lifting translates code into a higher-level form that can be verified and then regenerated [5, 22], with the lifted specification proved equivalent to the source. Wagner et al. compile a separate, verification-friendly build [60]. Instrumentation can make programs easier for automated tools to verify [1]. Ivy restructures protocols into decidable fragments by hand [42, 43]. Predicting solver cost is studied for choosing among tools [16, 27].

A specification proved equivalent to the source keeps the legal set inside as-if of that source. IntentCompile generates candidates from the current implementation and the contract. Theory-Lower rewrites toward a sketch V can decide, and V admits against the contract. A measurement

of O may not need that proof, but the compiler still ranks only among admitted programs so a cheaper but illegal candidate is not emitted.

Data Layout in Compilers. Production compilers restructure data when they can prove the change invisible [18, 25, 62]. MEMOIR makes collections explicit in IR [34]. Annotation-based AoS-to-SoA conversions are trusted rather than checked [46].

Those passes are unexposed slack (§3.3): extra cells or a different layout the interface does not observe. IntentCompile can name that slack and logical-parameter slack, a capacity the caller can observe, in one contract, and V admits the refactor.

Objectives. Objectives that rank programs come in many kinds. Autotuners such as PetaBricks rank by measured runtime [2]. McCabe's cyclomatic complexity [33] and Martin's coupling and abstraction [32] are computed from the program text. O in Section 3.5 is that supplied ranking over programs the contract admits. IntC fills it with a profile-guided measurement, wall-clock time on a named driver.

9 Discussion and Future Work

Writing Φ. Decoupling search from P_1 moves the specification burden onto the developer. That cost has limited adoption of contract-based tools. A contract inferred from the baseline tends to freeze incidental choices, such as a capacity of exactly 1000, and removes the slack the problem depends on. Contracts have to come from intent, not from the code. Translating a goal-oriented requirements model into *Pre*, *Post*, and $\mathcal{I}$ is future work. One route is a KAOS refinement of a goal tree down to atomic requirements in a formal specification language [59].

Under-Specification. The legal set is the slack in Φ_{func}, including slack the developer did not intend. A FIFO ring whose contract omits order admits a faster queue that reorders packets, and V will prove it. Checking a proposed contract against the baseline catches contracts that are too strong, not contracts that are too weak [38]. A missing requirement is a proved program the developer did not want. When a deployment or a rejected candidate surfaces a gap, that requirement should become a contract clause that later compilations re-check, so the same regression cannot return unless the requirements change. Completing Φ that way is the developer's cycle around the compiler, not a pass of Section 4. The same holds for a cost that is missing from $\mathcal{B}$ or O: concurrent false sharing on a padded ring cannot reject a candidate until it is named. Closing gaps permanently is the central open problem for intent compilation.

Temporarily Relaxing Constraints. Loosening *Pre* or *Post* enlarges $\text{Sat}(\Phi_{\text{func}})$, so the search can admit a program the tight contract would reject. Putting those clauses back makes that program illegal, and V has to run again. Loosening $\mathcal{B}$ does not change legality: a candidate admitted under a looser memory ceiling can still satisfy Φ_{func} after the tight ceiling is restored, and no observable test fails. The compiler also bounds how much of Gen it explores and how long V may run; loosening those changes which candidates finish in a compile, not $\text{Sat}(\Phi_{\text{func}})$. Restoring a temporary relaxation is future work: re-running V, re-checking $\mathcal{B}$, or re-searching under the tight bounds.

Unbounded Verification. IntC's claim is $\models_K$. An unbounded proof of a stateful component is the invariant-and-abstraction obligation Ivy discharges by restructuring into a decidable fragment [42, 43]. Carrying that obligation on a candidate, rather than stopping at K, is future work.

Where to Spend Compilation Time. IntC runs V before it measures O, so a candidate can consume a timeout before a cheap profile would have dropped it. Other schedules over the same Gen include estimating O first, running a cheap $\models_K$ before a longer query, and keeping candidates and proofs as artifacts so a later compile reuses them.

A Score that Predicts V. IntC's TheoryLower score orders descent, and against CBMC time it has Spearman's $\rho = 0.11$. A score that prefers the rewrite expected to shorten the query has to be fit to this V, and predicting SMT and verifier runtime from goal or program structure is already studied [16, 27].

Language Models as Compiler Inputs. A model can be the untrusted Gen of Theorem 4.2 at compile time [13, 21, 57], or, for a deterministic compile, help write the catalogs, TheoryLower scores, and benchmarks an instance enumerates. It can also help with the specification burden of writing Φ by drafting Φ_{func} from a prose requirement [31, 58] as a candidate for V, and by helping write the baseline P_1. IntC is the prototype for a C instance; a model can help instantiate those inputs for another language.

Language Models as Developers. Professional developers commonly use AI tools on coding tasks: 51% report daily use in the 2025 Stack Overflow survey [56], including multi-turn conversations on real GitHub work [14]. Later rewrites in that setting often break requirements that earlier turns had already met [17, 61]. IntentCompile takes those requirements as a checkable Φ and the program as P_1, which a model can fill [24]. The next rewrite is then a compile against Φ. Named requirements change when intent changes, which is less often than the program is rewritten, so a human can maintain Φ while the model rewrites P.

10 Conclusion

An as-if compiler preserves the source, including incidental choices the requirements do not fix, so it may omit a cheaper legal program. IntentCompile takes a baseline and a checkable contract, generates candidates, and admits those a sound verifier proves. It ranks the admitted set by O under the same lowering backend that compiled P_1, and emits P_1 when no admitted candidate is cheaper. IntC is a prototype C instance of that restricted compiler.

On the motivating example, IntC admits the 1024-cell ring that MSVC `/O2` will not emit from a 1000-cell remainder. On extracts whose contracts name a benchmark, the emit is cheaper on O than P_1 or is P_1. The catalogs, the bound K, and writing Φ from intent are the limits of IntC.

Data-Availability Statement

An artifact accompanies this paper: IntC, a Python source-to-source compiler for C, the contracts and C extracts of Section 7, CBMC harnesses, and the script that rebuilds the catalog, ablation, and timed-contract tables. Admission is $\models_K$ with CBMC 6.11.0. Measurement and listings use MSVC 19.29 `/O2` on Windows. From `poc/`, after `pip install -e .`, `python reproduce.py` rebuilds those tables. Solver times are not bit-stable. Admit, timeout, and counterexample outcomes, and rewrite names, must match `ARTIFACT_README.md`. The artifact is https://github.com/LightShield/IntC.

References

[1] Jesper Amilon, Zafer Esen, Dilian Gurov, Christian Lidström, and Philipp Rümmer. 2023. Automatic Program Instrumentation for Automatic Verification. In *Computer Aided Verification (CAV 2023) (LNCS, Vol. 13966)*. doi:10.1007/978-3-031-37709-9_14

[2] Jason Ansel, Cy Chan, Yee Lok Wong, Marek Olszewski, Qin Zhao, Alan Edelman, and Saman Amarasinghe. 2009. PetaBricks: A Language and Compiler for Algorithmic Choice. In *PLDI*. 38–49. doi:10.1145/1542476.1542481

[3] Ralph-Johan Back and Joakim von Wright. 1998. *Refinement Calculus: A Systematic Introduction*. Springer. doi:10.1007/978-1-4612-1674-2

[4] Joshua Berne, Timur Doumler, Rostislav Khlebnikov, and Andrzej Krzemieński. 2025. *Contracts for C++ — Rationale*. Technical Report P2899R1. ISO/IEC JTC1/SC22/WG21. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2899r1.pdf

[5] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A. Seshia, and Alvin Cheung. 2024. Verified Code Transpilation with LLMs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. arXiv:2406.03003

[6] Michael Carbin, Deokhwan Kim, Sasa Misailovic, and Martin C. Rinard. 2012. Proving Acceptability Properties of Relaxed Nondeterministic Approximate Programs. In *PLDI*. 169–180. hdl:1721.1/72437.

[7] Edmund M. Clarke, Daniel Kroening, and Flavio Lerda. 2004. A Tool for Checking ANSI-C Programs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 2988)*. 168–176. doi:10.1007/978-3-540-24730-2_15

[8] ClickHouse, Inc. 2026. src/Common/HashTable/HashTable.h. https://github.com/ClickHouse/ClickHouse/blob/e9b645ed83ad3c07b64f09fbd014e8f770d8691b/src/Common/HashTable/HashTable.h. Commit e9b645ed, accessed 2026-10-05.

[9] Loris D'Antoni, Roopsha Samanta, and Rishabh Singh. 2016. Qlose: Program Repair with Quantitative Objectives. In *CAV (LNCS, Vol. 9780)*. 383–401. doi:10.1007/978-3-319-41540-6_21

[10] Eva Darulova and Viktor Kuncak. 2014. Sound Compilation of Reals. In *POPL*. 235–248. doi:10.1145/2535838.2535874

[11] Benjamin Delaware, Clément Pit-Claudel, Jason Gross, and Adam Chlipala. 2015. Fiat: Deductive Synthesis of Abstract Data Types in a Proof Assistant. In *POPL*. 689–700. doi:10.1145/2676726.2677006

[12] DPDK Project. 2026. rte_ring (lib/ring/rte_ring.c, rte_ring_core.h, rte_ring_elem_pvt.h). https://github.com/DPDK/dpdk/blob/49bb9a5f7ae51128de7132ffe265192413643bcb/lib/ring/rte_ring.c. Commit 49bb9a5f, accessed 2026-10-05.

[13] Xiangxin Fang, Jiaqin Kang, Rodrigo C. O. Rocha, Sam Ainsworth, and Lev Mukhanov. 2026. LLM-VeriOpt: Verification-Guided Reinforcement Learning for LLM-Based Compiler Optimization. In *CGO*.

[14] Huizi Hao, Kazi Amit Hasan, Hong Qin, Marcos Macedo, Yuan Tian, Steven H. H. Ding, and Ahmed E. Hassan. 2024. An Empirical Study on Developers' Shared Conversations with ChatGPT in GitHub Pull Requests and Issues. arXiv:2403.10468

[15] Peter Hawkins, Alex Aiken, Kathleen Fisher, Martin Rinard, and Mooly Sagiv. 2011. Data Representation Synthesis. In *Proc. PLDI*. 38–49. doi:10.1145/1993498.1993504

[16] Andrew Healy, Rosemary Monahan, and James F. Power. 2017. Predicting SMT Solver Performance for Software Verification. *Electronic Proceedings in Theoretical Computer Science (F-IDE 2016)* 240 (2017), 20–37. doi:10.4204/EPTCS.240.2

[17] Yonghui Huang, Lin Ma, Amjed Tahir, Qian Zhang, Liwen Xiao, and Lysa Xiao. 2026. Regression Accumulation in Multi-Turn LLM Programming Conversations. arXiv:2607.01855

[18] Robert Hundt, Sandya Mannarswamy, and Dhruva Chakrabarti. 2006. Practical Structure Layout Optimization and Advice. In *Proc. Int. Symp. on Code Generation and Optimization (CGO)*. 233–244. doi:10.1109/CGO.2006.29

[19] Yuka Ikarashi, Gilbert Louis Bernstein, Alex Reinking, Hasan Genc, and Jonathan Ragan-Kelley. 2022. Exocompilation for Productive Programming of Hardware Accelerators. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. 703–718. doi:10.1145/3519939.3523446

[20] ISO/IEC. 2023. *Working Draft, Standard for Programming Language C++*. Technical Report N4950. ISO/IEC JTC1/SC22/WG21. [intro.abstract], §4.1.2.

[21] Hailong Jiang, Feng Yu, Emran Hossain, Jianfeng Zhu, Mengfei Ren, Qiang Guan, and Chunwei Xia. 2026. Can Large Language Models Recover Semantic Optimization Opportunities That Compilers Miss? arXiv:2608.03983

[22] Shoaib Kamil, Alvin Cheung, Shachar Itzhaky, and Armando Solar-Lezama. 2016. Verified Lifting of Stencil Computations. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 711–726. doi:10.1145/2908080.2908117

[23] Maurice Karnaugh. 1953. The map method for synthesis of combinational logic circuits. *Transactions of the American Institute of Electrical Engineers, Part I: Communication and Electronics* 72, 5 (1953), 593–599.

[24] Shuvendu K. Lahiri. 2026. Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents. arXiv:2603.17150

[25] Chris Lattner and Vikram Adve. 2005. Automatic Pool Allocation: Improving Performance by Controlling Data Structure Layout in the Heap. In *Proc. PLDI*. 129–142. doi:10.1145/1065010.1065027

[26] Juneyoung Lee, Yoonseung Kim, Youngju Song, Chung-Kil Hur, Sanjoy Das, David Majnemer, John Regehr, and Nuno P. Lopes. 2017. Taming Undefined Behavior in LLVM. In *PLDI*. doi:10.1145/3062341.3062343

[27] Will Leeson and Matthew B. Dwyer. 2022. Algorithm Selection for Software Verification using Graph Neural Networks. arXiv:2201.11711

[28] Xavier Leroy. 2009. A Formally Verified Compiler Back-end. *Journal of Automated Reasoning* 43, 4 (2009), 363–446. doi:10.1007/s10817-009-9155-4

[29] Calvin Loncaric, Emina Torlak, and Michael D. Ernst. 2016. Fast Synthesis of Fast Collections. In *Proc. PLDI*. 355–368. doi:10.1145/2908080.2908122

[30] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*. doi:10.1145/3453483.3454030

[31] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2025. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. In *ICSE*. doi:10.1109/ICSE55347.2025.00129

[32] Robert C. Martin. 2000. Design Principles and Design Patterns. Object Mentor.

[33] Thomas J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837

[34] Tommy McMichen, Nathan Greiner, Peter Zhong, Federico Sossai, Atmn Patel, and Simone Campanoni. 2024. Representing Data Collections in an SSA Form. In *Proc. CGO*. 308–321. doi:10.1109/CGO57630.2024.10444817

[35] Meta Platforms. 2026. folly/ProducerConsumerQueue.h. https://github.com/facebook/folly/blob/bfae630e01e3d7087a9d30bb8fea9506690751f9/folly/ProducerConsumerQueue.h. Commit bfae630e; mask-based since commit 594a1f31 (2026-09-16); compare-and-reset through d15c1215 (2025-08-01). Accessed 2026-10-05.

[36] Bertrand Meyer. 1988. *Object-Oriented Software Construction*. Prentice Hall.

[37] Sasa Misailovic, Michael Carbin, Sara Achour, Zichao Qi, and Martin C. Rinard. 2014. Chisel: Reliability- and Accuracy-Aware Optimization of Approximate Computational Kernels. In *OOPSLA*. 309–328. doi:10.1145/2660193.2660231

[38] Md Rakib Hossain Misu, Iris Ma, and Cristina V. Lopes. 2026. Spec-Harness: Measuring and Improving Behavioral Adequacy of LLM-Synthesized Formal Specifications. arXiv:2604.00280

[39] Carroll Morgan. 1994. *Programming from Specifications* (2nd ed.). Prentice Hall.

[40] George C. Necula. 2000. Translation validation for an optimizing compiler. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation*. 83–94.

[41] Aina Niemetz, Mathias Preiner, and Yoni Zohar. 2024. Scalable Bit-Blasting with Abstractions. In *Computer Aided Verification (CAV 2024) (LNCS, Vol. 14681)*. 178–200. doi:10.1007/978-3-031-65627-9_9

[42] Oded Padon, Giuliano Losa, Mooly Sagiv, and Sharon Shoham. 2017. Paxos Made EPR: Decidable Reasoning about Distributed Protocols. *Proc. ACM Program. Lang.* 1, OOPSLA (2017), 108:1–108:31. doi:10.1145/3140568

[43] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. 2016. Ivy: Safety Verification by Interactive Generalization. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 614–630. doi:10.1145/2908080.2908118

[44] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 151–166.

[45] Xueying Qin, Liam O'Connor, and Michel Steuwer. 2023. Primrose: Selecting Container Data Types by Their Properties. *The Art, Science, and Engineering of Programming* 7, 3 (2023). doi:10.22152/programming-journal.org/2023/7/11

[46] Pawel K. Radtke and Tobias Weinzierl. 2025. Annotation-Guided AoS-to-SoA Conversions and GPU Offloading with Data Views in C++. *Concurrency and Computation: Practice and Experience* (2025). doi:10.1002/cpe.70199 arXiv:2502.16517.

[47] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 519–530. doi:10.1145/2491956.2462176

[48] Henry Gordon Rice. 1953. Classes of Recursively Enumerable Sets and Their Decision Problems. *Trans. Amer. Math. Soc.* 74, 2 (1953), 358–366. doi:10.1090/S0002-9947-1953-0053041-6

[49] Martin Rinard. 2026. Quantitative Comparison of Credible Compilation and Verification In Coding Agent Compiler Development. arXiv:2605.08927

[50] Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Gratian Lup, Jubi Taneja, and John Regehr. 2017. Souper: A Synthesizing Superoptimizer. arXiv:1711.04422

[51] Eric Schkufza, Rahul Sharma, and Alex Aiken. 2014. Stochastic Optimization of Floating-Point Programs with Tunable Precision. In *PLDI*. 53–64. doi:10.1145/2594291.2594302

[52] Rahul Sharma, Eric Schkufza, Berkeley Churchill, and Alex Aiken. 2015. Conditionally Correct Superoptimization. In *OOPSLA*. doi:10.1145/2858965.2814278

[53] Stelios Sidiroglou-Douskos, Sasa Misailovic, Henry Hoffmann, and Martin Rinard. 2011. Managing Performance vs. Accuracy Trade-offs with Loop Perforation. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. 124–134. doi:10.1145/2025113.2025133

[54] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *ASPLOS*. 404–415. doi:10.1145/1168857.1168907

[55] Saurabh Srivastava, Sumit Gulwani, and Jeffrey S. Foster. 2010. From Program Verification to Program Synthesis. In *POPL*. 313–326. doi:10.1145/1706299.1706337

[56] Stack Overflow. 2025. Developer Survey 2025: AI. https://survey.stackoverflow.co/2025/ai. Accessed 2026-10-13.

[57] Jubi Taneja, Avery Laird, Cong Yan, Madan Musuvathi, and Shuvendu K. Lahiri. 2025. LLM-Vectorizer: LLM-Based Verified Loop Vectorizer. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO)*. 137–149. doi:10.1145/3696443.3708929

[58] Shubham Ugare, Tarun Suresh, Sasa Misailovic, and Julien Vanegue. 2026. Fun2spec: Code Contract Synthesis At Scale. In *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*

(FSE Industry Track). doi:10.1145/3803437.3805262
[59] Axel van Lamsweerde. 2001. Goal-Oriented Requirements Engineering: A Guided Tour. In *Proceedings of the 5th IEEE International Symposium on Requirements Engineering*. 249–263. doi:10.1109/ISRE.2001.948567
[60] Jonas Wagner, Volodymyr Kuznetsov, and George Candea. 2013. -OVERIFY: Optimizing Programs for Fast Verification. In *14th Workshop on Hot Topics in Operating Systems (HotOS XIV)*. USENIX Association. https://www.usenix.org/system/files/conference/hotos13/hotos13-final69.pdf
[61] Lu Yan, Xuan Chen, and Xiangyu Zhang. 2026. When the Specification Emerges: Benchmarking Faithfulness Loss in Long-Horizon Coding Agents. arXiv:2603.17104
[62] Peng Zhao, Shimin Cui, Yaoqing Gao, Raúl Silvera, and José Nelson Amaral. 2007. Forma: A Framework for Safe Automatic Array Reshaping. *ACM Trans. Program. Lang. Syst.* 30, 1, Article 2 (2007). doi:10.1145/1290520.1290522

A Proofs

Throughout, "result" means the observable part of $[\![P]\!](\sigma)$, and $Acc_\theta(\sigma)$ is the set of observable results that $Post_\theta$ accepts on σ.

Proposition 3.5 (As-If Programs Remain Legal). If $P \models \Phi_{\text{func}}$, $P' \approx P$, and $\theta_{P'} = \theta_P$, then $P' \models \Phi_{\text{func}}$.

Proof. Suppose $P' \not\models \Phi_{\text{func}}$. Then either $\theta_{P'} \notin \mathcal{I}$, or some $\sigma \in Pre$ is a non-terminating, undefined, or unacceptable run of P'. The first case is impossible because $\theta_{P'} = \theta_P \in \mathcal{I}$. In the second case, $P' \approx P$ gives the same observable result for P on σ, so P also violates the contract, a contradiction. □

Proposition 3.4 (Legal Set Expands Exactly on Observable Slack). Assume local patching, and that each $Post_\theta$ with $\theta \in \mathcal{I}$ is realized by some program. Let $P \models \Phi_{\text{func}}$. Then $\text{Sat}(\Phi_{\text{func}})$ contains a program that is not observationally equivalent to P iff the contract has input, output, or logical-parameter slack, counting only results some program produces. Unexposed slack is not in that list: it need not change those results.

Proof. Assume also that all results below are produced by some program. Let $o_P(\sigma)$ be P's result on σ.

($\Leftarrow$) *Input slack.* Let $\sigma_0 \notin Pre$ and let o be a producible result different from $o_P(\sigma_0)$. Patch P at σ_0. The patch agrees with P on Pre, so it satisfies the contract, and differs from P on σ_0.

Output slack. Let $\sigma_0 \in Pre$ and $\theta = \theta_P$ with two accepted results, one of them $o \neq o_P(\sigma_0)$. (If the slack is only at another parameter, it is the logical-parameter case.) Patch P at σ_0 to produce o.

Logical-parameter slack. Let $\sigma_0 \in Pre$ and $\theta_1, \theta_2 \in \mathcal{I}$ with $Acc_{\theta_1}(\sigma_0) \neq Acc_{\theta_2}(\sigma_0)$. Both sets are nonempty. Some accepted result o at one of those parameters differs from $o_P(\sigma_0)$. Start from a program for that parameter and patch it at σ_0.

($\Rightarrow$) If there is no such slack, then $Pre = \Sigma$, each $Acc_\theta(\sigma)$ has at most one producible element, and those elements coincide for all $\theta \in \mathcal{I}$. That element is $o_P(\sigma)$. Any legal P' must produce it everywhere, so $P' \approx P$. □

Unexposed slack is omitted because it need not change observable API results.

Lemma 4.1 (Lowering Preserves a Proved Contract). If $P^* \models \Phi_{\text{func}}$ (respectively $\models_K$), then $\mathcal{L}_{\text{Target}}(P^*) \models \Phi_{\text{func}}$ (respectively $\models_K$).

Proof. Let $B = \mathcal{L}_{\text{Target}}(P^*)$. Parameters of B are those of the source given to the backend. If P^* satisfies Φ_{func} (or $\models_K$) then it has no undefined behavior on the covered inputs (or traces), terminates with an accepted result, and the compiler correctness assumption copies that result to B. Proposition 3.5 is the unbounded case. □

Theorem 4.2 (Soundness of a Restricted Intent Compiler). Let V be sound for $\models$ or for $\models_K$, and let the compiler emit source P^* and binary $\mathrm{Bin}^* = \mathcal{L}_{\mathrm{Target}}(P^*)$.

(1) If $P^* \neq P_1$, then P^* is in $\mathrm{Gen}(P_1, R, D)$, V proved it, and it passed $\mathcal{B}$. Hence $P^* \models \Phi_{\mathrm{func}}$ or $P^* \models_K \Phi_{\mathrm{func}}$, matching V, and the same judgment holds of Bin^* by Lemma 4.1.
(2) The measured cost of Bin^* is at most that of $\mathcal{L}_{\mathrm{Target}}(P_1)$, when measurement runs.
(3) If $P_1 \models \Phi_{\mathrm{func}}$ (or $\models_K$), then Bin^* satisfies the same judgment in every case.

Proof. Winner selection (Algorithm 6) returns P_1 or a member of $\mathcal{A}$. If $P^* \neq P_1$, then $P^* \in \mathcal{A}$, so $P^* \in \mathrm{Gen}$, V answered proved, and $\mathcal{B}$ passed. Soundness of V gives $\models$ or $\models_K$. Lemma 4.1 transfers the judgment to the binary. Measured cost is minimized over a set containing $\mathcal{L}_{\mathrm{Target}}(P_1)$. If $P^* = P_1$ and P_1 already satisfies the judgment, apply Lemma 4.1 to P_1. □

B Pipeline Algorithms

This appendix gives each stage of IntentCompile as an algorithm: inputs, outputs, the promises that algorithm makes, and a proof of those promises. Every stage claims termination, relative to its oracles returning (the solver for $\mathcal{I}$, V, the checker for $\mathcal{B}$, lowering, and measurement). Parameters such as the TheoryLower score, the CEGIS distance, the attempt cap, the checker for $\mathcal{B}$, and the rewrite catalogs are inputs of the algorithms. Section 6 instantiates them. Hole domains are finite. V returns proved, a counterexample, or unknown, and respects its time budget. A rewrite in a catalog has a syntactic applicability condition and optional side constraints on holes. Nothing below claims that $\mathrm{Sat}(\Phi_{\mathrm{func}})$ is searchable, that the catalogs are complete, or that V decides every query.

B.1 CheckRealiz

Algorithm 1 (CheckRealiz). Input: $\Phi_{\mathrm{func}} = \langle Pre, Post, \mathcal{I} \rangle$ and a solver for $\mathcal{I}$ that answers satisfiable, unsatisfiable, or unknown. A time budget on that solver, when present, is mapped to unknown. Output: continue, or a stop (vacuous, unsat with a core, or unknown).

(1) If $Pre = \emptyset$, report vacuous and stop.
(2) Ask the solver whether $\mathcal{I}$ is satisfiable.
(3) If the answer is unsatisfiable, report unsat with the solver's unsatisfiable core and stop.
(4) If the answer is unknown, stop.
(5) If the answer is satisfiable, report continue.

Lemma B.1 (CheckRealiz). *Assume the solver for $\mathcal{I}$ is sound.*

(1) Termination. *If the solver for $\mathcal{I}$ returns in finite time, so does the algorithm. A timeout is* unknown *and is a return.*
(2) No candidate. *The algorithm emits no program.*
(3) Vacuous stop. *If it reports* vacuous*, then $Pre = \emptyset$, so every program satisfies Φ_{func} and the contract states no intent.*
(4) Unsat stop. *If it reports* unsat*, then $\mathcal{I}$ is empty, so no program satisfies Φ_{func}.*
(5) Unknown stop. *An* unknown *answer is a stop, not a proof that $\mathcal{I}$ is inhabited or empty.*
(6) Continue. *A* continue *result is only that $\mathcal{I}$ has a model. It is not a candidate, and it does not decide whether Post is realizable on Pre.*

Proof. Steps 1 and 3–5 are local. The only query is step 2. If the solver returns satisfiable, unsatisfiable, or unknown, one of those steps returns, which is (1). The algorithm has no step that constructs or admits a program, so (2) holds. Step 1 returns vacuous exactly when $Pre = \emptyset$. Satisfaction of Φ_{func} quantifies over Pre, so an empty precondition is true of every program, which is (3). Step 3 returns unsat only on a sound unsat answer, which means $\mathcal{I}$ has no valuation. A program that satisfies Φ_{func} must have $\theta_P \in \mathcal{I}$, so no such program exists, which is (4). Step 4

returns on unknown without a model or a core, which is (5). The query in step 2 mentions $\mathcal{I}$ only. A model of $\mathcal{I}$ does not produce a program and does not prove that every $\sigma \in Pre$ has an accepted result under $Post$, which is (6). □

The stronger query $\forall\sigma \in Pre.\ \exists o.\ (\sigma, o) \in Post_\theta$ is not this stage. An unsatisfiable harness is discovered later, when no candidate is proved. Termination is relative to the solver: the stage does not decide an arbitrary theory of $\mathcal{I}$, and a solver with no budget may diverge.

B.2 TheoryLower

Let *score* map sketches to $\delta \cdot \mathbb{N}$ for some granularity $\delta > 0$, and let τ be a threshold at which rewriting stops. Both are inputs. The intended reading is a weighted count of constructs outside a chosen cheap set for the source language and V, so descent aims at that set. Any score with those values for which every applied rewrite strictly decreases the score makes the lemma hold. IntC's score is the weighted feature count of Section 6. $score(S) \le \tau$ does not imply that V decides S.

Algorithm 2 (TheoryLower). Input: baseline P_1, catalog R, *score*, and threshold τ. Output: a sketch S tagged below-threshold or stuck, or no sketch.

(1) Set $S \leftarrow P_1$. Rewrites, not this step, introduce holes.
(2) If $score(S) \le \tau$, return no sketch, tagged below-threshold.
(3) Among pairs of a rewrite $r \in R$ and a site of S where r applies, choose one that minimizes $score(r(S))$. If no pair gives a strict decrease: if a rewrite has already been applied, return (S, stuck); otherwise return no sketch, tagged stuck.
(4) Set $S \leftarrow r(S)$. If $score(S) \le \tau$, return $(S, \text{below-threshold})$. Otherwise go to step 3.

Lemma B.2 (TheoryLower). (1) Termination. *The algorithm returns after at most* $score(P_1)/\delta$ *rewrites.*
(2) Catalog. *Every rewrite it applies is from R, at a site that matches that rewrite. Holes come from those rewrites.*
(3) Exits. *CEGIS receives a sketch only if a rewrite applied. That sketch has score* $\le \tau$*, or no remaining rewrite in R lowers the score. If no rewrite applied, there is no TheoryLower seed.*
(4) Unverified. *No returned sketch is submitted to V at this stage.*

Proof. Each applied rewrite lowers *score* by at least δ, and the score is nonnegative, so that can happen at most $score(P_1)/\delta$ times, which is (1). Step 3 selects from R at matching sites, and step 1 does not punch holes, which is (2). Step 2 returns no sketch. Step 3 returns a sketch only if one was already applied, or no sketch. Step 4 returns a rewritten sketch, which is (3). No step calls V, which is (4). □

The stage makes no semantic claim. The sketch may violate the contract for every hole value, and later stages decide which fillings survive. The loop is greedy: a rewrite that first raises the score and enables a larger drop later is never taken. A stuck result does not abort GeneratePool, which starts from P_1.

B.3 Baseline-Directed CEGIS

Let $S[h]$ be a sketch with holes ranging over a finite domain D, and let *dist* be a distance on D. Closeness is relative to *dist*. IntC minimizes distance of baseline holes to θ_{P_1} and extra holes subject to the side constraints (Section 6).

Algorithm 3 (Baseline-Directed CEGIS). Input: sketch S, domain D, side constraints, $\mathcal{I}$, baseline parameters θ_{P_1}, verifier V with budget t, and a finite attempt cap N. Output: a filled program $S[h]$, or fail.

(1) Set $R_{\text{ex}} \leftarrow \emptyset$ and $n \leftarrow 0$.
(2) If $n = N$, return fail.
(3) Find $h \in D \setminus R_{\text{ex}}$ that meets the side constraints, gives $\theta_{S[h]} \in \mathcal{I}$, and minimizes $dist(h, \theta_{P_1})$. If none exists, return fail.
(4) Run $V(S[h], \Phi_{\text{func}}, t)$.
(5) If the answer is proved, return $S[h]$.
(6) If the answer is a counterexample, add h to R_{ex}. Do not add the failing input as a constraint on later proposals. Set $n \leftarrow n + 1$ and go to step 2.
(7) If the answer is unknown, return fail.

Theorem 5.1 (Properties of Baseline-Directed CEGIS).. Let $S[h]$ range over a finite hole domain D, and let N be the attempt cap ($N = |D|$ if there is no cap).

(1) If CEGIS returns $S[h]$, then V proved $S[h]$.
(2) *Termination.* If V returns on each query, CEGIS returns after at most $\min(N, |D|)$ iterations. Step 3 searches a finite set, so it returns.
(3) If some $h^\circ \in D$ makes $S[h^\circ]$ satisfy the contract (or $\models_K$), V never answers unknown, and there is no attempt cap, then CEGIS returns a satisfying assignment closest to θ_{P_1}.

Proof. (1) The only success return is step 5, which requires proved.

(2) V is assumed to return (proved, a counterexample, or unknown) because it respects t. Each unsuccessful iteration that does not fail on unknown adds one h to R_{ex} and never retries it. The loop also stops when $n = N$. Together that is at most $\min(N, |D|)$ iterations. Step 3 enumerates a finite remainder of D.

(3) Let $H^\circ = \{h' \in D \mid S[h'] \models \Phi_{\text{func}}\}$ (or $\models_K$), nonempty by assumption. Soundness of V keeps every member of H° out of R_{ex}, because only a counterexample excludes a valuation. Step 3 therefore always sees a nonempty feasible set, so it does not fail. Since V never answers unknown and there is no cap, (2) ends the run on proved. The returned h is feasible for step 3 at that iteration, hence minimizes $dist(\cdot, \theta_{P_1})$ over a set that still contains H°. An attempt cap preserves (1) and (2) and may drop (3). □

The bound $|D|$ is a worst case. Side constraints shrink the search. The guarantee covers only the holes of the given sketch. If no filling of that sketch satisfies the contract, CEGIS fails, and a different structure is a rewrite, not a closer h.

B.4 BudgetNormalize

Algorithm 4 (BudgetNormalize). Input: a finite set of CEGIS seeds, baseline P_1, and a terminating checker for $\mathcal{B}$. Output: a (possibly empty) subset of the seeds, and P_1 unmarked as a seed. GeneratePool is not skipped.

(1) Apply the checker for $\mathcal{B}$ to each seed and to P_1.
(2) Keep those seeds for which the checker passes. Drop the rest.
(3) If none remain, record that the seed set is empty.
(4) Return the kept set. Do not disable GeneratePool.

Lemma B.3 (BudgetNormalize). (1) Termination. *If the checker terminates on each of finitely many programs, so does the algorithm.*
(2) Filter. *It adds no program.*
(3) Kept seeds. *Every kept seed still carries whatever V proof it had, and passes $\mathcal{B}$.*
(4) Baseline. *The checker is applied to P_1, but P_1 is never added to the kept seeds, so a miss does not remove it from* (6).

(5) Empty seeds. *An empty kept set does not skip* GeneratePool.

Proof. One checker run per seed and one for P_1 is finite if each run is, which is (1). The algorithm only retains or drops members of its input, which is (2). A kept seed is a subset of the input, so a prior proved answer remains, and step 2 is exactly the checker for $\mathcal{B}$, which is (3). Step 1 runs the checker on P_1 and step 2 never adds it to the kept set, which is (4). Step 4 returns after recording emptiness and does not disable the next stage, which is (5). □

The stage does not repair a seed that misses $\mathcal{B}$. A later rewrite in GeneratePool may still produce a different program that fits.

B.5 GeneratePool

Let R_{pool} be a finite catalog of representation rewrites, disjoint in purpose from the TheoryLower catalog though both are subsets of the generator's R.

Algorithm 5 (GeneratePool). Input: baseline P_1, catalog R_{pool}, any seeds that survived BudgetNormalize, verifier V, and the checker for $\mathcal{B}$. Output: a finite set C of programs.

(1) Set C to the surviving seeds (empty if none).
(2) For each rewrite $r \in R_{\text{pool}}$ that applies to P_1, form the sketch $r(P_1)$, run Algorithm 3 on it, and if CEGIS returns a program that V proved and that passes $\mathcal{B}$, add that program to C.
(3) Return C. Do not rank by O or by an estimated cost.

Lemma B.4 (GeneratePool). (1) Termination. *The algorithm terminates if each CEGIS run does and the checker for $\mathcal{B}$ returns on each tried program.*
(2) Starts from P_1. *Step 2 runs even when the seed set is empty, and even when TheoryLower returned* stuck.
(3) Admission. *Every program in the returned C is in $\mathcal{A}$. Surviving seeds are already in $\mathcal{A}$.*
(4) No ranking. *The algorithm does not consult O.*

Proof. R_{pool} is finite. Each CEGIS run terminates by Theorem 5.1(2). Each budget check is one run of a terminating checker, which is (1). Step 2 iterates that catalog from P_1 with no guard on the seed set or on TheoryLower's tag, which is (2). A program added in step 2 was obtained from P_1 by a rewrite from the catalog, filled by CEGIS, proved by V, and passed $\mathcal{B}$, hence lies in $\mathcal{A}$. Surviving seeds were proved by an earlier CEGIS run and passed BudgetNormalize, which is (3). Step 3 forbids ranking, and no other step reads O, which is (4). □

The forward pass can miss a combination that pays only after a later rewrite. Exploring sequences of rewrites would find those combinations at a cost exponential in sequence length. Ranking remains (6).

B.6 Winner Selection

Algorithm 6 (Winner Selection). Input: P_1, the set C from Algorithm 5, backend $\mathcal{L}_{\text{Target}}$, and objective O. Output: source P^* and binary $\text{Bin}^* = \mathcal{L}_{\text{Target}}(P^*)$.

(1) Let $\mathcal{A}$ be C (every member already proved and inside $\mathcal{B}$).
(2) If the contract names no O, return the first member of $\mathcal{A}$, or P_1 if $\mathcal{A}$ is empty.
(3) Lower P_1 and every member of $\mathcal{A}$ with $\mathcal{L}_{\text{Target}}$.
(4) Measure each binary on the named workload and hardware, and return the source whose binary minimizes O, preferring P_1 on ties, as in (6). If $\mathcal{A}$ is empty, return P_1.

Lemma B.5 (Winner Selection). (1) Termination. *If lowering and measurement return on each of finitely many programs, so does the algorithm. If there is no O, there is no measurement loop.*

(2) *It returns P_1 or a member of $\mathcal{A}$.*
(3) *Measurement is not admission: a program not proved by V is not ranked. With no O, the first admitted program is emitted.*

The remaining claims are Theorem 4.2.

Proof. $\mathcal{A}$ is finite by Lemma B.4. With no O, step 2 returns immediately, which is (1). Otherwise one lowering and one measurement per member of $\mathcal{A} \cup \{P_1\}$ is finite if each call returns, which is (1). The arg min set is $\{\mathcal{L}_{\text{Target}}(P) \mid P \in \mathcal{A} \cup \{P_1\}\}$, or $\{P_1\}$ when $\mathcal{A}$ is empty, or the first of $\mathcal{A}$ when there is no O, so the returned source is P_1 or a member of $\mathcal{A}$, which is (2). Steps 2 and 4 rank only admitted programs, which is (3). □